# 29-Working with Pandas DataFrame

## Introduction

- Lecture focused on practical operations with Pandas DataFrames using a City Bike trip data example.
- Interactive session included demonstrations, coding exercises, and Q&A.

## Dataset Overview

- City Bike trip data contains ~2,000 rows and 10 columns such as trip duration, start/stop time, station info, bike ID, user type, etc.

## Indexing & Slicing

- Access columns with brackets: e.g., `df['start station name']`.
- Use `.head()` and `.tail()` for viewing the first/last n rows.
- Retrieve multiple columns: pass a list of column names.
- Use `.value_counts()` to see frequency of values in a column.
- Access specific cell values via `{column}[row_index]` or `.at[row, column]`.

## Filtering Data

- Apply conditions: `df[df['trip duration'] > 200]` filters rows where duration > 200.

- Combine conditions using logical operators for more complex filters.

## Index Types and Usage (.loc/.iloc)

- `.loc`: Used for explicit (user-defined) indices; format is `.loc[row, column]`.

- `.iloc`
  : Used for implicit (default, zero-based) indices; both row and column referenced by position.

- When changing DataFrame indices, be aware of which method to use.

- Safe to chain and combine `.loc` with conditions for subsetting.

## File Import Operations

- Best practice: Keep datasets in the same folder as your Jupyter Notebook/script for simplified pathing.

- Demonstrated both relative and absolute file path usages with `pd.read_csv()`.

- Discussed file system navigation for importing datasets.

## Adding & Deleting Columns and Rows

- **Add Column:** Assign an array (of correct length) to a new or existing column.

- Supports default or varying values (e.g., using `np.nan` for missing data).

- **Delete Column:** Use `.drop(columns=[...])` or `del df['col']`
  ; note difference between creating new DataFrame vs. modifying in place.

- **Add Row:**
  Assign a list of values to a new row index (typically at the end); explained reindexing for maintaining consecutive numbering.

- **Delete Row(s):** Use `.drop(index=[list])`, and remember to reassign or use `inplace=True` for persistent changes.

## Sorting Data

- **Sort by Index:** `.sort_index()`
  arranges rows/columns by index labels (row: axis=0, column: axis=1).

- **Sort by Values:** `.sort_values(by=...)`
  supports single or multiple columns; allows ascending/descending and in-place options.

## Iterating Over DataFrames

- **.itertuples():** Fastest, returns tuples with row data, index as the first element.

- **.iterrows():** Returns (index, Series) pairs for each row.

- **.items():** Iterates over columns, yields (column name, Series) pairs.

- Outputs and ideal use-cases for each method demonstrated.

## Handling Binary Data with Pickle

- Covered how to load and iterate over binary files using `pickle.load()`.

- Warning: repeated loading moves the cursor, so save the data to a variable first when filtering or conditional checks are needed.

## Assignment & Q&A

- Discussed assignment deadlines and extension policies for Pandas/Numpy exercises.

- Mentioned upcoming assignment uploads (including for RegEx).

- Addressed common student questions regarding DataFrame operations, assignment clarifications, and error troubleshooting.

---

### Additional Notes

- Key distinctions: `.loc` (label-based), `.iloc` (position-based).

- In-place vs. out-of-place operations: Use `inplace=True` to avoid creating copies.

- Use `np.where()` for fetching row indexes meeting a condition.

- For adding data with `np.nan`, all columns become float type by upcasting.

- Sorting, adding, deleting, and iteration methods all demonstrated through examples and debugged in live coding.